

Laboratorio IA 4: Percepcion local

Materia: Inteligencia Artificial

Dinamica: practica en terminal **Entregable:** experimento, codigo y conclusiones

Transformar el robot de la clase 3 en un agente reactivo. La poblacion ya no evoluciona una ruta completa: evoluciona una politica que observa las celdas vecinas y decide el siguiente movimiento durante la ejecucion.

1. De la clase 3 a la clase 4

En la clase 3 cada individuo era una secuencia fija de 18 comandos. El robot recibia todo el plan antes de moverse y no podia cambiarlo al encontrar un obstaculo. Era un agente *open loop*.

En esta version conservamos la misma cuadrícula, inicio, meta y algoritmo genetico, pero cambiamos la representacion. El robot observa antes de cada movimiento y consulta una regla de su ADN. Ahora tenemos un agente *closed loop*.

La evolucion ya no busca una ruta concreta para este recorrido. Busca una politica: un conjunto de reglas que relaciona una observacion con una accion.

2. Preparar y ejecutar

Ejecuta el programa desde la carpeta del laboratorio:

```
python3 robot_percepcion.py
python3 robot_percepcion.py --semilla 7 --sin-pausa
```

Cada variante guarda automaticamente su ultima poblacion en un archivo con el formato `generacionN_identific`. El valor N depende de la semilla y de cuando encuentra la meta. El archivo contiene las politicas de todos los individuos, no solo la mejor. Cada politica muestra su ADN y la tabla de reglas `observacion ->accion`; por ejemplo, `0000 ->D`.

Este ejemplo completo genera una poblacion y recupera exactamente el archivo que acaba de crear:

```
python3 robot_percepcion.py --modo mutacion --semilla 7 \\  
  --sin-pausa --identificador experimento_a  
archivo=$(ls -t generacion*_experimento_a.txt | head -n 1)  
python3 robot_percepcion.py --cargar "$archivo" \\  
  --ejecutar mejor --sin-pausa
```

Para cargar esa misma generacion y ejecutar todos sus individuos:

```
python3 robot_percepcion.py --cargar "$archivo" \\  
  --ejecutar todos --sin-pausa
```

La opcion **-no-guardar** permite ejecutar un experimento sin crear un nuevo archivo de generacion.

Tambien puedes comparar las variantes por separado:

```
python3 robot_percepcion.py --modo mutacion --semilla 7 --sin-pausa
python3 robot_percepcion.py --modo cruce --semilla 7 --sin-pausa
```

3. Como percibe el robot

El sensor local observa las cuatro celdas vecinas en este orden: U, D, L, R. Una celda fuera del tablero o con obstaculo se codifica como 1; una celda libre como 0.

Por tanto existen $2^4 = 16$ observaciones posibles. El ADN tiene 16 genes. El indice de una observacion selecciona el gen que indica la accion que se intentara ejecutar.

Observacion	Significado	Accion del gen
0000	Ningun vecino bloqueado	<code>adn[0]</code>
0001	Solo derecha bloqueada	<code>adn[1]</code>
1010	Arriba e izquierda bloqueadas	<code>adn[10]</code>
1111	Los cuatro vecinos bloqueados	<code>adn[15]</code>

El robot no recibe la posicion exacta de la meta ni una ruta precalculada. Solo usa la observacion local para consultar su politica.

4. Que se conserva de la clase 3

- La cuadrícula de 8 por 8, el inicio, la meta y un entorno con obstaculos.
- La funcion de fitness: distancia, pasos utiles, intentos bloqueados, repeticiones y llegada.
- La poblacion, el elitismo, la mutacion y el cruce de un punto.
- La comparacion entre solo mutacion y mutacion mas cruce.

Lo que cambia es el significado del ADN. En la clase 3, cada posicion era el siguiente movimiento de una ruta. Ahora cada posicion es una regla reutilizable para una observacion.

5. Experimento guiado

1. Ejecuta el modo comparativo con las semillas 7, 21 y 42.
2. Registra llegada, generaciones, puntaje, choques o intentos bloqueados, pasos utiles y diversidad.
3. Observa si la politica llega a la meta o repite un ciclo.
4. Compara mutacion y cruce con el mismo presupuesto de generaciones.
5. Cambia una sola penalizacion del fitness y repite una semilla.

Semilla	Metodo	Llego	Generaciones	Puntaje	Bloqueos	Diversidad
7	Solo mutacion					
7	Mutacion + cruce					
21	Solo mutacion					
21	Mutacion + cruce					
42	Solo mutacion					
42	Mutacion + cruce					

1. Que diferencia hay entre evolucionar una ruta y evolucionar una politica?
2. Por que una politica local puede entrar en un ciclo?
3. Que informacion pierde el robot al no conocer su posicion exacta?
4. El cruce combina rutas completas o reglas de percepcion? Que efecto puede tener?
5. Una politica que funciona en este mapa funcionaria necesariamente en otro mapa?

6. Reto de modificacion

Realiza obligatoriamente las cuatro modificaciones siguientes y conserva las demas condiciones del experimento. Documenta el efecto de cada una:

- agrega la direccion relativa de la meta a la observacion;
- incorpora una memoria de la ultima accion;
- cambia el numero maximo de pasos y explica los ciclos observados;
- compara la misma politica en una segunda distribucion de obstaculos.

7. Entrega

Entrega el archivo `robot_percepcion.py`, la tabla con tres semillas y una conclusion de 8 a 10 lineas. La conclusion debe responder: *que gano el robot al percibir y que limitaciones introdujo una politica basada solo en vecinos?*

8.Codigo completo

El codigo se inserta directamente desde el archivo de la practica para mantener sincronizados el manual y el programa.

```
import argparse
import json
import os
import random
import re
import time

# =====
# 1. CONFIGURACION DEL PROBLEMA
# =====
COMANDOS = ("U", "D", "L", "R")
ORDEN_SENSORES = ("U", "D", "L", "R")

TAMANO = 8
INICIO = (0, 0)
META = (7, 7)

OBSTACULOS = {
    (0, 3), (1, 3), (2, 0), (2, 2), (2, 3),
    (4, 5), (5, 2), (5, 3), (5, 4),
    (6, 6),
}

# El ADN tiene una regla para cada patron posible de cuatro sensores.
LONGITUD_ADN = 16
PASOS_MAXIMOS = 18
```

```

TAMANO_POBLACION = 100
ELITE = 10
PADRES = 40

EMOJI_ROBOT = "\U0001f916"
EMOJI_META = "\U0001f7e9"
EMOJI_META_ALCANZADA = "\U0001f389"
EMOJI_OBSTACULO = "\u2b1b"
EMOJI_INICIO = "\U0001f535"
EMOJI_LIBRE = "\u2b1c"

# =====
# 2. PERCEPCION Y SIMULACION DEL ENTORNO
# =====

def limpiar_pantalla():
    """Limpia la terminal antes de dibujar el siguiente paso."""
    os.system("cls" if os.name == "nt" else "clear")

def destino_de(posicion, comando):
    """Devuelve la celda que queda en la direccion indicada."""
    fila, columna = posicion
    cambios = {"U": (-1, 0), "D": (1, 0), "L": (0, -1), "R": (0, 1)}
    cambio_fila, cambio_columna = cambios[comando]
    return fila + cambio_fila, columna + cambio_columna

def esta_bloqueada(posicion):
    """Indica si una celda no puede ser ocupada por el robot."""
    fila, columna = posicion
    return not (0 <= fila < TAMANO and 0 <= columna < TAMANO) or posicion in OBSTACULOS

def observar(posicion):
    """Codifica como bits las celdas bloqueadas en orden U, D, L y R."""
    patron = 0
    for comando in ORDEN_SENSORES:
        patron <<= 1
        patron |= int(esta_bloqueada(destino_de(posicion, comando)))
    return patron

def decidir(adn, posicion):
    """Consulta la regla del ADN correspondiente a la observacion actual."""
    return adn[observar(posicion)]

def mover(posicion, comando):
    """Intenta ejecutar un comando y devuelve posicion y resultado."""
    destino = destino_de(posicion, comando)
    if not (0 <= destino[0] < TAMANO and 0 <= destino[1] < TAMANO):
        return posicion, "borde"
    if destino in OBSTACULOS:
        return posicion, "obstaculo"
    return destino, "avance"

def recorrer(adn):
    """Ejecuta una politica reactiva y registra su trayectoria y metricas."""
    posicion = INICIO
    trayectoria = [posicion]
    observaciones = []
    acciones = []
    choques = 0

```

```

pasos_utiles = 0

for _ in range(PASOS_MAXIMOS):
    observacion = observar(posicion)
    comando = decidir(adn, posicion)
    nueva_posicion, resultado = mover(posicion, comando)
    observaciones.append(observacion)
    acciones.append(comando)
    if resultado in ("borde", "obstaculo"):
        choques += 1
    if resultado == "avance":
        pasos_utiles += 1
    posicion = nueva_posicion
    trayectoria.append(posicion)
    if posicion == META:
        break

return trayectoria, choques, pasos_utiles, observaciones, acciones

# =====
# 3. REGLA DE SUPERVIVENCIA: EL FITNESS
# =====
def evaluar(adn):
    """Calcula el fitness de una politica: cuanto mayor, mejor solucion."""
    trayectoria, choques, pasos_utiles, _, _ = recorrer(adn)
    posicion = trayectoria[-1]
    distancia = abs(META[0] - posicion[0]) + abs(META[1] - posicion[1])
    visitas_repetidas = len(trayectoria) - len(set(trayectoria))

    puntaje = 500
    puntaje -= distancia * 35
    puntaje += pasos_utiles * 4
    puntaje -= choques * 30
    puntaje -= visitas_repetidas * 3

    if posicion == META:
        puntaje += 2000
    return puntaje

def diversidad(poblacion):
    """Cuenta cuantas politicas diferentes hay en la poblacion."""
    return len({"".join(adn) for adn in poblacion})

def reglas_de_politica(adn):
    """Devuelve la politica con sus observaciones binarias como claves."""
    return {
        f"{indice:04b}": comando
        for indice, comando in enumerate(adn)
    }

# =====
# 4. OPERADORES GENETICOS
# =====
def mutar(adn, tasa_mutacion, rng):
    """Cambia algunas reglas al azar segun la tasa de mutacion."""
    return [
        rng.choice(COMANDOS) if rng.random() < tasa_mutacion else comando
        for comando in adn
    ]

```

```

def cruzar(primer_padre, segundo_padre, rng):
    """Combina el comienzo de una politica y el final de otra."""
    punto = rng.randint(1, len(primer_padre) - 1)
    return primer_padre[:punto] + segundo_padre[punto:]

# =====
# 5. CICLO EVOLUTIVO
# =====

def evolucionar(tasa_mutacion, usar_cruce, semilla, maximo_generaciones=3000):
    """Ejecuta el algoritmo genetico y devuelve la mejor politica."""
    rng = random.Random(semilla)
    poblacion = [
        [rng.choice(COMANDOS) for _ in range(LONGITUD_ADN)]
        for _ in range(TAMANO_POBLACION)
    ]
    historial = []

    for generacion in range(maximo_generaciones + 1):
        poblacion.sort(key=evaluar, reverse=True)
        mejor = poblacion[0]
        puntaje = evaluar(mejor)
        trayectoria, choques, pasos utiles, _, _ = recorrer(mejor)
        historial.append((generacion, puntaje, diversidad(poblacion)))

        if trayectoria[-1] == META:
            return resultado(
                usar_cruce, mejor, generacion, puntaje, diversidad(poblacion),
                choques, pasos utiles, historial, poblacion,
            )

        nueva_poblacion = [robot[:] for robot in poblacion[:ELITE]]
        while len(nueva_poblacion) < TAMANO_POBLACION:
            primer_padre = rng.choice(poblacion[:PADRES])
            if usar_cruce:
                segundo_padre = rng.choice(poblacion[:PADRES])
                hijo = cruzar(primer_padre, segundo_padre, rng)
            else:
                hijo = primer_padre[:]
            nueva_poblacion.append(mutar(hijo, tasa_mutacion, rng))
        poblacion = nueva_poblacion

        poblacion.sort(key=evaluar, reverse=True)
        mejor = poblacion[0]
        trayectoria, choques, pasos utiles, _, _ = recorrer(mejor)
        return resultado(
            usar_cruce, mejor, maximo_generaciones, evaluar(mejor),
            diversidad(poblacion), choques, pasos utiles, historial, poblacion,
        )

def resultado(usar_cruce, adn, generacion, puntaje, diversidad_final,
             choques, pasos utiles, historial, poblacion):
    """Reune las metricas que compararemos en el experimento."""
    trayectoria, _, _, _, _ = recorrer(adn)
    return {
        "metodo": "mutacion + cruce" if usar_cruce else "solo mutacion",
        "adn": adn,
        "generacion": generacion,
        "puntaje": puntaje,
        "diversidad": diversidad_final,
        "choques": choques,
        "pasos utiles": pasos utiles,
        "llego": trayectoria[-1] == META,
        "historial": historial,
    }

```

```

        "poblacion": [robot[:] for robot in poblacion],
    }

def guardar_generacion(resultado_actual, identificador, semilla):
    """Guarda la ultima poblacion en un archivo de texto estructurado."""
    identificador_limpio = re.sub(r"[^A-Za-z0-9_-]+", "_", identificador)
    nombre = f"generacion{resultado_actual['generacion']}_{identificador_limpio}.txt"
    datos = {
        "version": 2,
        "generacion": resultado_actual["generacion"],
        "identificador": identificador_limpio,
        "semilla": semilla,
        "metodo": resultado_actual["metodo"],
        "longitud_adn": LONGITUD_ADN,
        "orden_sensores": "U D L R",
        "poblacion": [
            {
                "indice": indice,
                "adn": "".join(robot),
                "reglas": reglas_de_politica(robot),
            }
            for indice, robot in enumerate(resultado_actual["poblacion"])
        ],
    }
    with open(nombre, "w", encoding="utf-8") as archivo:
        json.dump(datos, archivo, indent=2)
    return nombre

def cargar_generacion(ruta):
    """Carga y valida una poblacion guardada previamente."""
    with open(ruta, encoding="utf-8") as archivo:
        datos = json.load(archivo)

    poblacion = datos.get("poblacion")
    if not isinstance(poblacion, list) or not poblacion:
        raise ValueError("El archivo no contiene una poblacion valida.")
    if datos.get("longitud_adn") != LONGITUD_ADN:
        raise ValueError("La longitud del ADN no coincide con esta version.")
    politicas = [
        individuo["adn"] if isinstance(individuo, dict) else individuo
        for individuo in poblacion
    ]
    if any(
        not isinstance(adn, str)
        or len(adn) != LONGITUD_ADN
        or any(comando not in COMANDOS for comando in adn)
        for adn in politicas
    ):
        raise ValueError("El archivo contiene politicas invalidas.")

    return {
        "generacion": datos.get("generacion", "desconocida"),
        "identificador": datos.get("identificador", os.path.basename(ruta)),
        "metodo": datos.get("metodo", "generacion cargada"),
        "poblacion": [list(adn) for adn in politicas],
    }

def resultado_de_politica(adn, metodo, generacion, poblacion):
    """Construye las metricas necesarias para animar una politica cargada."""
    trayectoria, choques, pasos utiles, _, _ = recorrer(adn)
    return {
        "metodo": metodo,

```

```

        "adn": adn,
        "generacion": generacion,
        "puntaje": evaluar(adn),
        "diversidad": diversidad(poblacion),
        "choques": choques,
        "pasos_utiles": pasos_utiles,
        "llego": trayectoria[-1] == META,
        "historial": [],
        "poblacion": [robot[:] for robot in poblacion],
    }

def ejecutar_poblacion(datos, modo_ejecucion):
    """Selecciona el mejor individuo o devuelve todos para animarlos."""
    poblacion = datos["poblacion"]
    poblacion_ordenada = sorted(poblacion, key=evaluar, reverse=True)
    if modo_ejecucion == "mejor":
        return [resultado_de_politica(
            poblacion_ordenada[0], datos["metodo"], datos["generacion"], poblacion
        )]
    return [resultado_de_politica(
        adn, datos["metodo"], datos["generacion"], poblacion
    ) for adn in poblacion]

# =====
# 6. VISUALIZACION Y CLI
# =====

def dibujar(resultado, pausa):
    """Anima la trayectoria decidida por la politica reactiva."""
    adn = resultado["adn"]
    trayectoria, _, _, observaciones, acciones = recorrer(adn)

    for paso, posicion in enumerate(trayectoria[1:], start=1):
        limpiar_pantalla()
        print(
            f"{{resultado['metodo'].upper()}} | "
            f"Gen {{resultado['generacion']}} | Puntos: {{resultado['puntaje']}}"
        )
        print(
            f"{{EMOJI_ROBOT}} = robot | {{EMOJI_META_ALCANZADA}} = meta alcanzada | "
            f"{{EMOJI_META}} = meta | {{EMOJI_OBSTACULO}} = obstaculo"
        )
        print(f"{{EMOJI_INICIO}} = inicio | {{EMOJI_LIBRE}} = celda libre\n")
        for fila in range(TAMANO):
            linea = ""
            for columna in range(TAMANO):
                celda = (fila, columna)
                if celda == posicion:
                    linea += EMOJI_ROBOT
                elif celda == INICIO:
                    linea += EMOJI_INICIO
                elif celda == META:
                    linea += EMOJI_META_ALCANZADA if posicion == META else EMOJI_META
                elif celda in OBSTACULOS:
                    linea += EMOJI_OBSTACULO
                else:
                    linea += EMOJI_LIBRE
            print(linea)
        print(
            f"Paso {{paso}}/{{len(trayectoria)} - 1} | "
            f"Observacion: {{observaciones[paso - 1]:04b}} | "
            f"Accion: {{acciones[paso - 1]}} | Politica: {''.join(adn)}"
        )
    if pausa:

```

```

        time.sleep(pausa)

def mostrar_resultado(resultado, pausa):
    """Muestra la animacion y las metricas finales de una variante."""
    dibujar(resultado, pausa)
    estado = "LLEGO" if resultado["llego"] else "NO LLEGO"
    print(f"\nResultado: {estado}")
    print(f"Pasos utiles: {resultado['pasos_utiles']}")
    print(f"Diversidad final: {resultado['diversidad']}/{TAMANO_POBLACION}")
    if pausa:
        time.sleep(1)

def mostrar_poblacion(resultados, pausa):
    """Anima todos los individuos guardados y muestra un resumen final."""
    for indice, resultado_actual in enumerate(resultados, start=1):
        print(f"\nIndividuo {indice}/{len(resultados)}")
        dibujar(resultado_actual, pausa)
    print(f"\nSe ejecutaron {len(resultados)} individuos en el mapa.")

def main():
    """Lee las opciones de terminal y ejecuta el experimento solicitado."""
    parser = argparse.ArgumentParser(
        description="Compara mutacion y cruce en politicas con percepcion local."
    )
    parser.add_argument(
        "--modo",
        choices=("comparar", "mutacion", "cruce"),
        default="comparar",
    )
    parser.add_argument("--semilla", type=int, default=7)
    parser.add_argument("--sin-pausa", action="store_true")
    parser.add_argument(
        "--identificador",
        help="Texto usado en el nombre generacionN_identificador.txt.",
    )
    parser.add_argument(
        "--cargar",
        metavar="ARCHIVO",
        help="Carga una generacion guardada en lugar de evolucionar.",
    )
    parser.add_argument(
        "--ejecutar",
        choices=("mejor", "todos"),
        default="mejor",
        help="Al cargar, ejecuta el mejor individuo o toda la poblacion.",
    )
    parser.add_argument(
        "--no-guardar",
        action="store_true",
        help="No guarda la poblacion final despues de evolucionar.",
    )
    args = parser.parse_args()

    pausa = 0 if args.sin_pausa else 0.08
    if args.cargar:
        datos = cargar_generacion(args.cargar)
        resultados = ejecutar_poblacion(datos, args.ejecutar)
        if args.ejecutar == "todos":
            mostrar_poblacion(resultados, pausa)
        else:
            mostrar_resultado(resultados[0], pausa)
    return

```

```

experimentos = {
    "mutacion": ("solo mutacion", False),
    "cruce": ("mutacion + cruce", True),
}
modos = tuple(experimentos) if args.modo == "comparar" else (args.modo,)
resultados = []

for indice, modo in enumerate(modos):
    _, usar_cruce = experimentos[modo]
    resultado_actual = evolucionar(
        tasa_mutacion=0.08,
        usar_cruce=usar_cruce,
        semilla=args.semilla + indice,
    )
    resultados.append(resultado_actual)
    if not args.no_guardar:
        identificador = args.identificador or (
            f"{modo}_semilla{args.semilla + indice}"
        )
        nombre = guardar_generacion(
            resultado_actual, identificador, args.semilla + indice
        )
        print(f"Generacion guardada en: {nombre}")
    if args.ejecutar == "todos":
        mostrar_poblacion(
            ejecutar_poblacion(
                {
                    "generacion": resultado_actual["generacion"],
                    "metodo": resultado_actual["metodo"],
                    "poblacion": resultado_actual["poblacion"],
                },
                "todos",
            ),
            pausa,
        )
    else:
        mostrar_resultado(resultado_actual, pausa)

if len(resultados) == 2:
    print("\nCOMPARACION")
    print("Metodo          Llego  Generaciones  Puntaje  Choques  Diversidad")
    print("-----")
    for resultado_actual in resultados:
        llego = "si" if resultado_actual["llego"] else "no"
        print(
            f"{resultado_actual['metodo']:<20}{llego:<7}"
            f"{resultado_actual['generacion']:<15}"
            f"{resultado_actual['puntaje']:<9}"
            f"{resultado_actual['choques']:<9}"
            f"{resultado_actual['diversidad']}/{TAMANO_POBLACION}"
        )

if __name__ == "__main__":
    main()

```